Design and Analysis of a 32x32-bit SRAM
Digital VLSI Design Project

A Hasitha
ABNP Sirisha
Sasidhar Varadha
Chiranjeevi.K

Contents

1 Introduction **3**

2 Individual Component Analysis **3**
2.1 Buffer Circuit 3
2.1.1 Inverter 3
2.2 2 Input NAND 6
2.3 3 Input NAND 7
2.4 2 Input NOR 8
2.5 6T SRAM Cell 10

3 32x32 SRAM Architecture **10**

4 Adder Design **10**
4.1 Optimised Design 11
4.2 Latency and Performance 13
4.3 Power Consideration 14
4.4 Unoptimized Design 15

5 Decoder Design **16**
5.1 Optimised Design 16
5.2 Latency and Performance 17
5.3 Power Consideration 18

6 32x32 SRAM Array **18**
6.1 Precharge Circuit 19
6.2 Optimised Design 19
6.3 6T SRAM Cell Design 19
6.4 MOSFET Sizing for SRAM Cell 20
6.5 Read Static Noise Margin 21
6.6 Write Static Noise Margin 23

7 Sense Amplifier Design **25**
7.1 Design 25
7.2 Simulation Results and Delay Calculations 26
7.3 Power Consideration 27
7.4 Trade-offs 28

8 Complete 32x32 bit SRAM Design **28**
8.1 Optimised Complete Design 28
8.2 Latency and Performance 29
8.3 Power Consideration 30

9 Conclusion **30**

1 Introduction

Memory arrays are an essential building block of all digital systems. We will design an SRAM array that contains 32 x 32-bit words using the given tech node 65nm technology. In order to support operation as a FIFO, the memory is accessed using a 5-bit address.

Block Level Description:

- **Adder:** Accepts two 5-bit address inputs (addra(4:0), addrb(4:0)) along with a carry-in (Cin). Produces a 5-bit summed address (a(4:0)) that is forwarded to the decoder.
- **Decoder:** Translates the 5-bit summed address into one-hot word line signals (W0–W31). This ensures that exactly one row of the SRAM array is activated at a time.
- **SRAM Array(32x32):** A memory block with 32 words, each 32 bits wide. The active word line connects the selected row to the differential bit line pairs (b0–b31, b'0–b'31).
- **Sense Amplifier:** Detects the small voltage difference on each bit line pair and amplifies it to produce full-swing digital outputs (out0–out31).

2 Individual Component Analysis

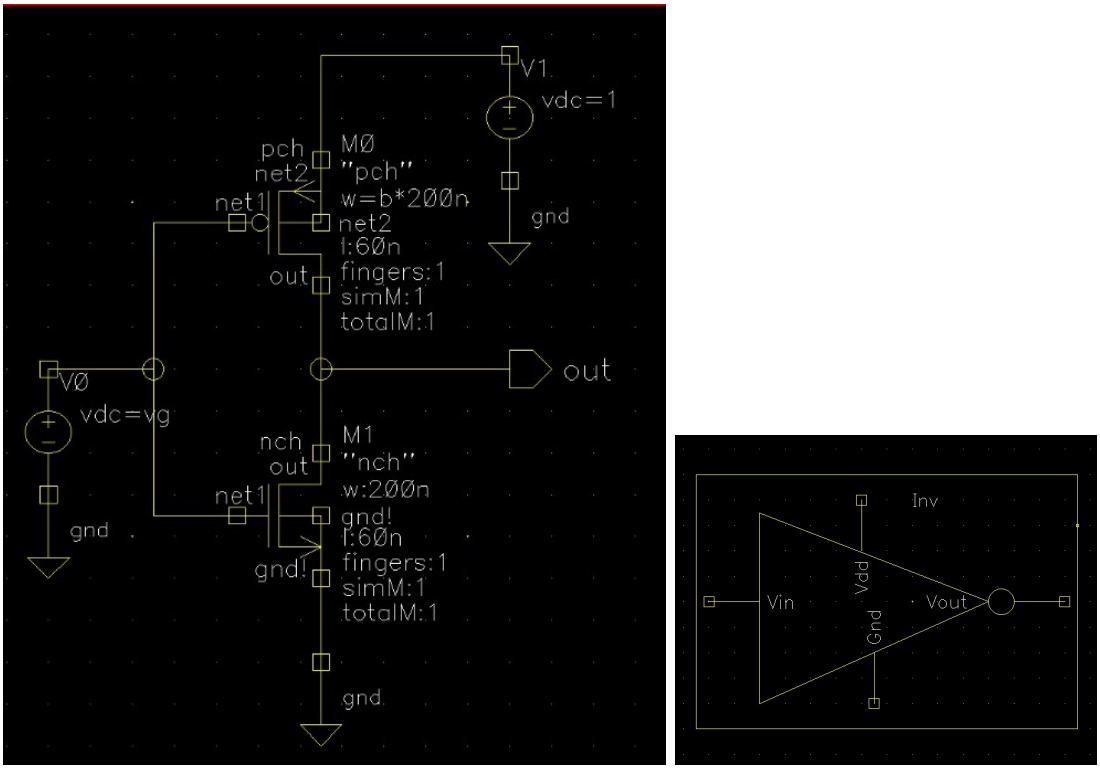
2.1 Buffer Circuit

2.1.1 Inverter

To obtain the inverter threshold, we perform dc analysis. We sweep the input from 0V to VDD, plot the output against it, and look for the point where Vin = Vout = Vinv which is the inverter threshold. To find the size of PMOS for which $V_{inv} = V_{DD}/2$, we define the width of PMOS as a variable and run a parametric simulation.

Inverter Threshold:
The required β ratio is obtained through simulations.

$$\beta = \frac{(W/L)_p}{(W/L)_n}$$



(a) CMOS Inverter Schematic (b) Symbol of CMOS Inverter

Figure 1: CMOS Inverter: Schematic and Symbol

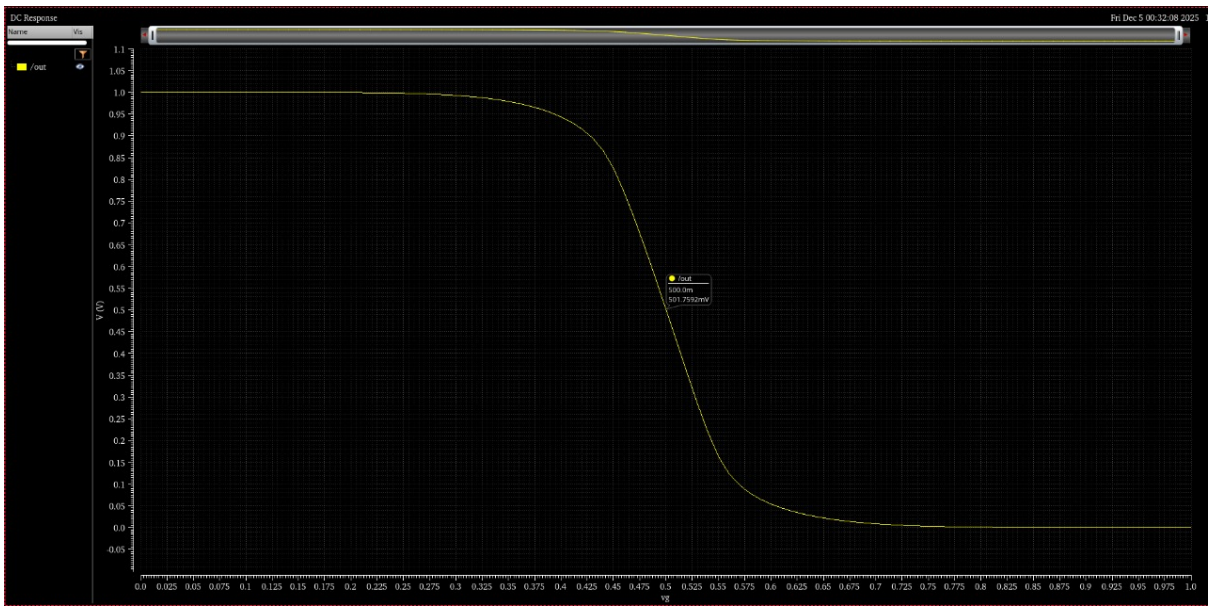


Figure 2: Inverter Voltage Transfer Characteristic (VTC) and Threshold Point

We get equal noise margin when $\beta = 2.35$.

We use a FO4 inverter circuit to characterise all the inverter delays. 5 Inverters are cascaded.

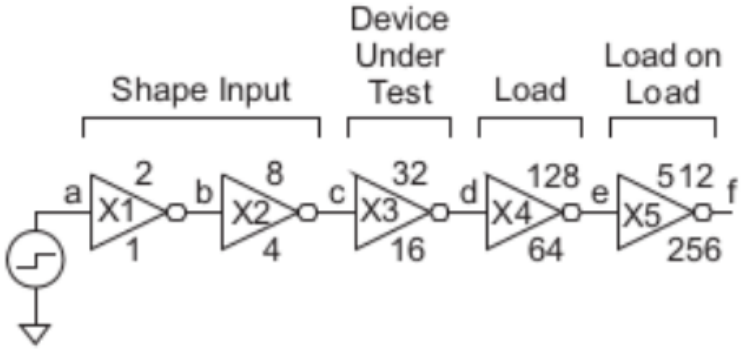
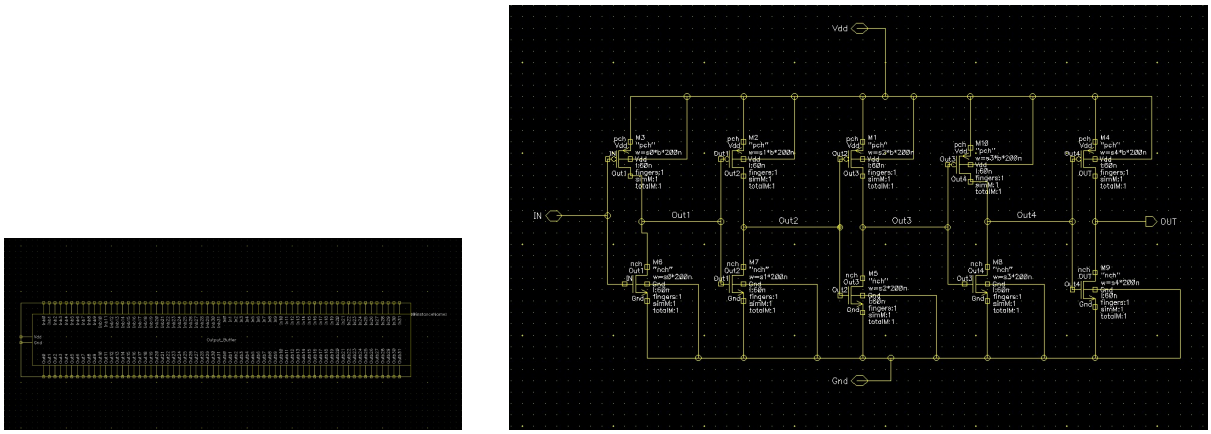


Figure 3: Inverter FO4 Circuit

X3 is our device under test. X1 and X2 shape the input, and the last two stages act as the load. The input capacitance of X4 depends not only on its C_{gs} but also on C_{gd}. The gate–drain capacitance C_{gd} is Miller-multiplied as node e switches, and would be effectively doubled if node e were to switch instantaneously. However, when node e is loaded with X5, it switches at a slower and more realistic rate, which slightly reduces the effective capacitance presented at node d by X4.

Transient analysis is performed to compute the delays. A pulsating input is applied to X1. The propagation delay is measured from the output of X2 (which serves as the input to X3) to the output of X3. The width of the PMOS transistor is scaled to obtain equal propagation delays for both input transitions: low $\rightarrow$ high and high $\rightarrow$ low.



(a) Buffer Symbol (b) Buffer Schematic

Figure 4: Buffer: Symbol and Schematic

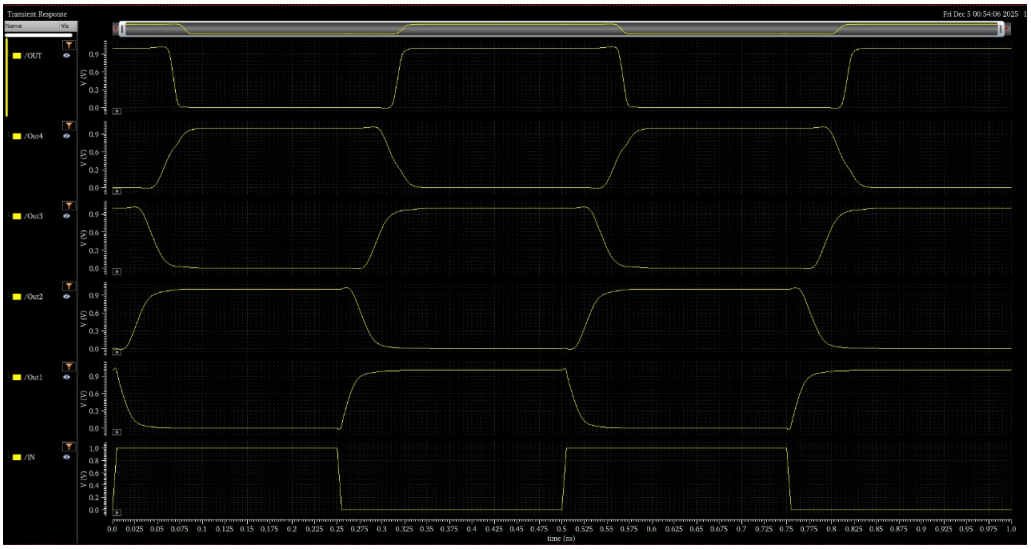
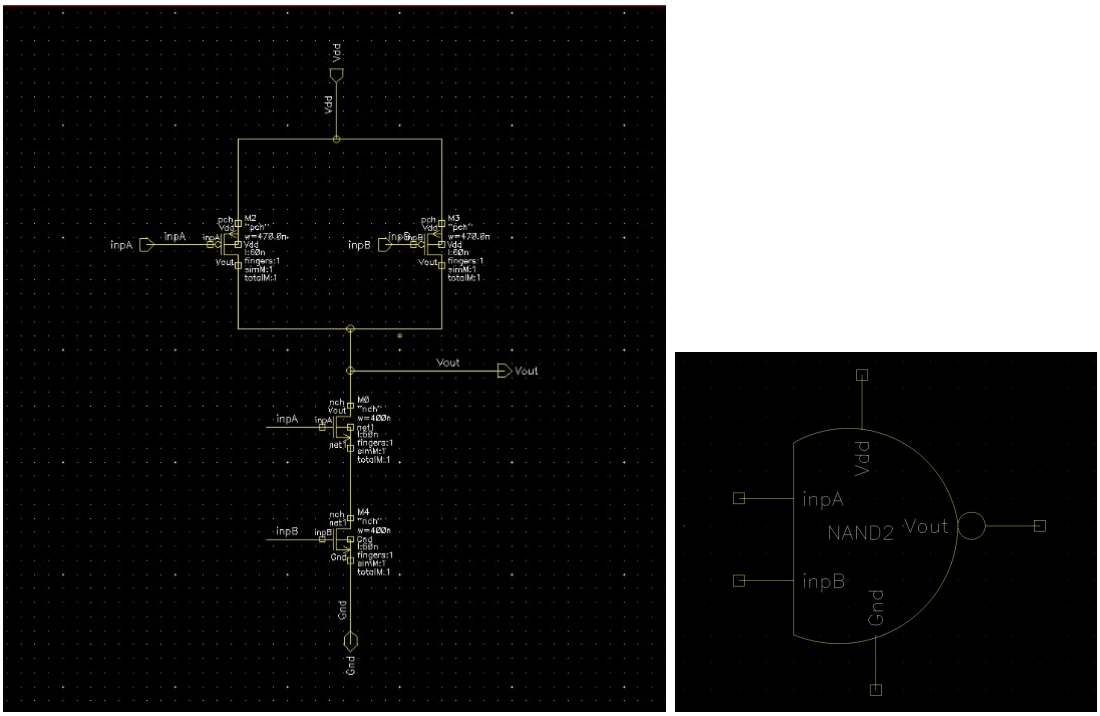


Figure 5: Simulation Waveforms of Buffer Circuit

2.2 2 Input NAND

CMOS logic is used to build the 2-input NAND gate. The NMOS transistors are scaled twice since they are connected in series, so the net resistance reduces by half. The PMOS sizing is designed for equal propagation delays. The delays are simulated using transient analyses.

- **Schematic:**



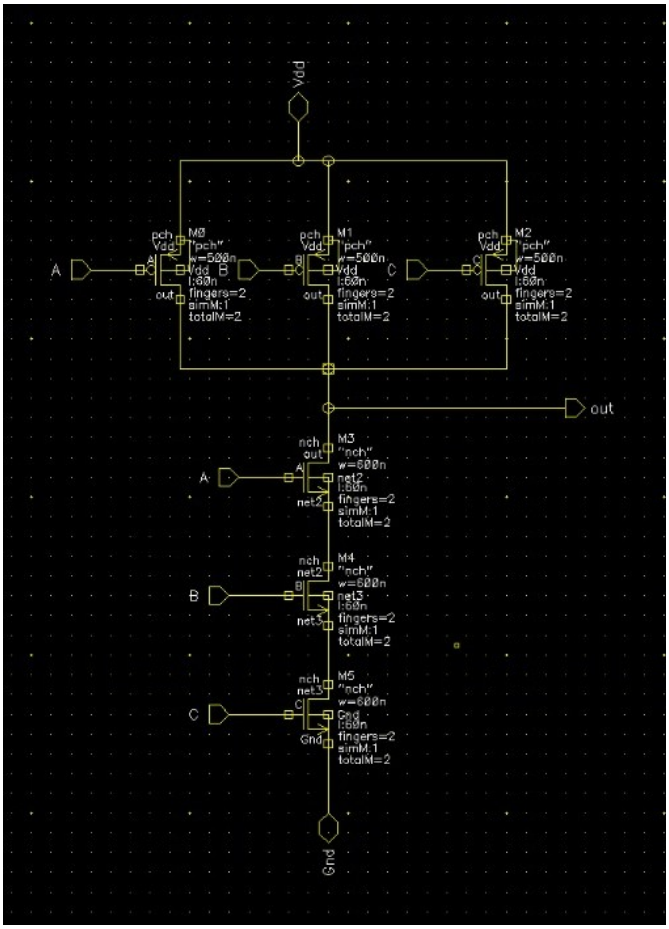
(a) NAND2 Schematic (b) NAND2 Symbol

Figure 6: 2-Input NAND Gate: Schematic and Symbol

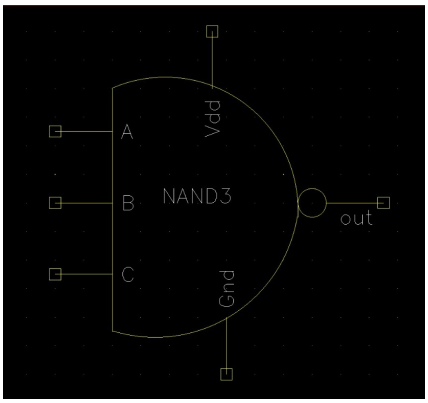
2.3 3 Input NAND

CMOS logic is used to build the 3-input NAND gate. The NMOS transistors are scaled thrice since they are connected in series, so the net resistance reduces to a third. The PMOS sizing is designed for equal propagation delays. The delays are simulated using transient analyses.

- **Schematic:**



(a) NAND3 Schematic



(b) NAND3 Symbol

Figure 7: 3-Input NAND Gate: Schematic and Symbol

- **Simluation Results:**

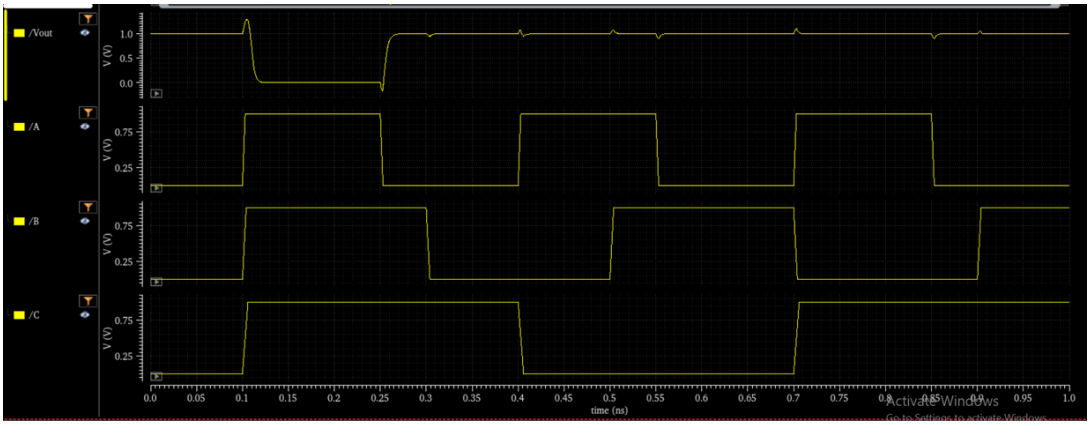


Figure 8: Simulation Waveforms of 3-Input NAND Gate

- **Latency**

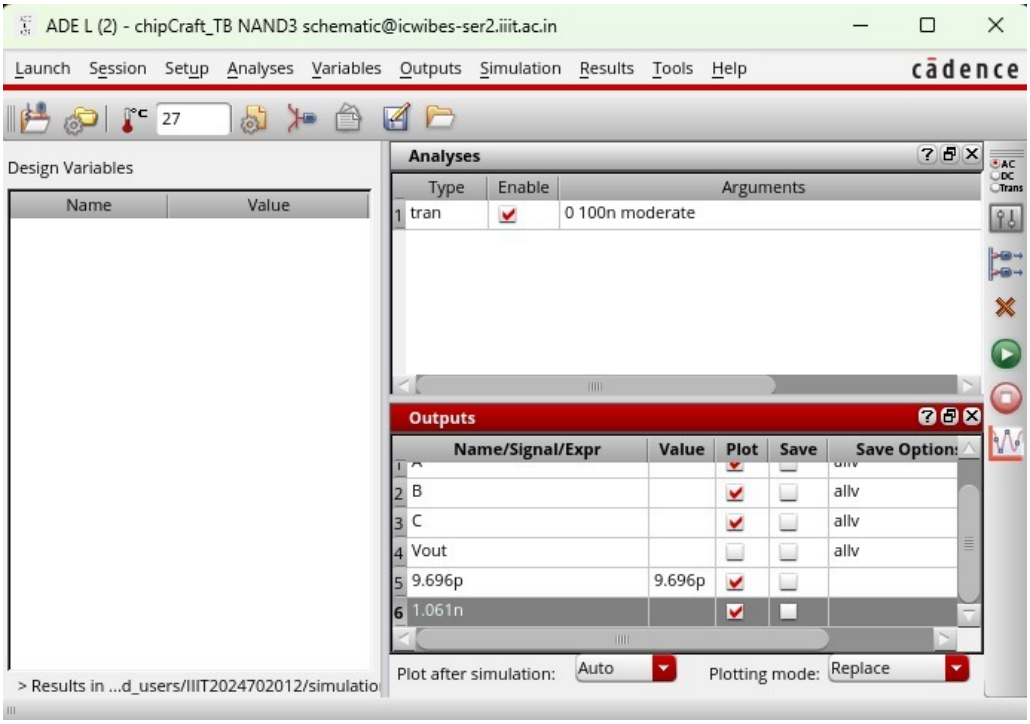
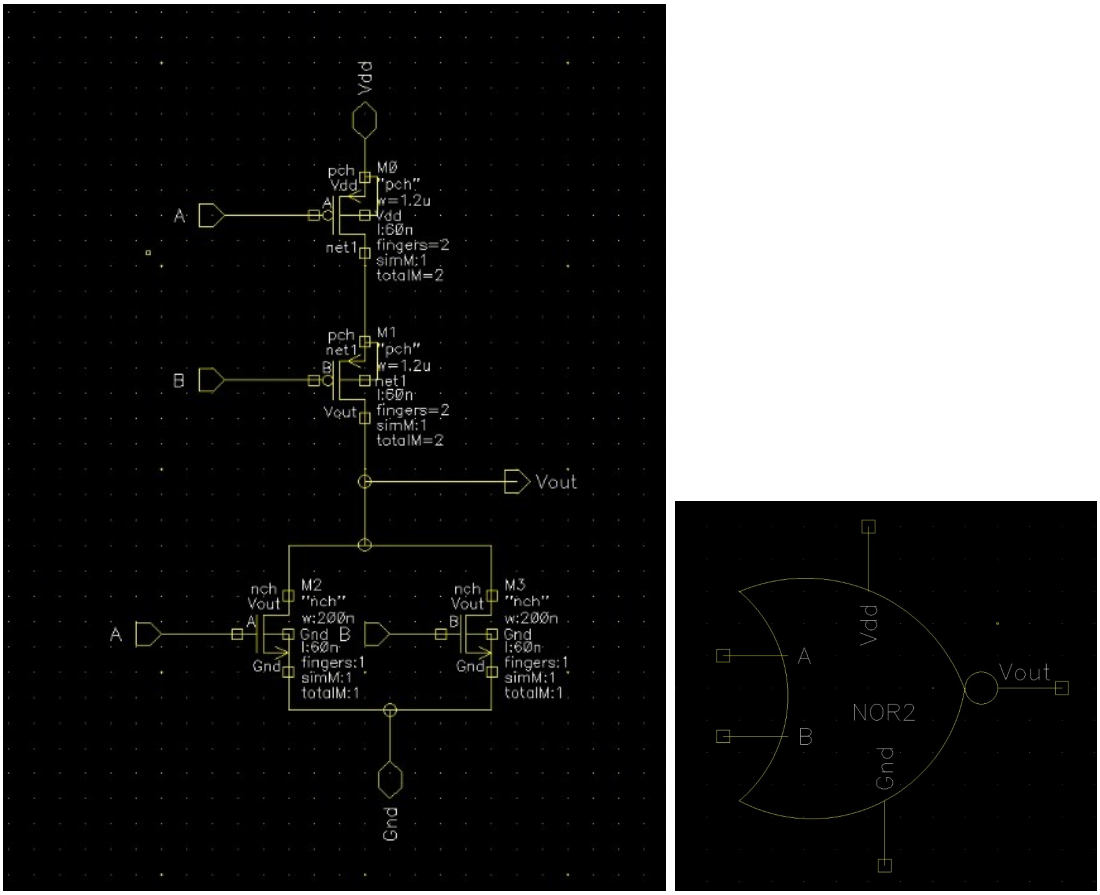


Figure 9: Propagation Delay Characteristics of 3-Input NAND Gate

2.4 2 Input NOR

CMOS logic is used to build the 2-input NOR gate. The NMOS transistors are sized to minimum (W/L) as they are connected in parallel. The PMOS sizing is designed for equal propagation delays for the worst-case. The delays are simulated using transient analyses.

- **Schematic:**



(a) NOR2 Schematic (b) NOR2 Symbol

Figure 10: 2-Input NOR Gate: Schematic and Symbol

- **Simluation Results:**

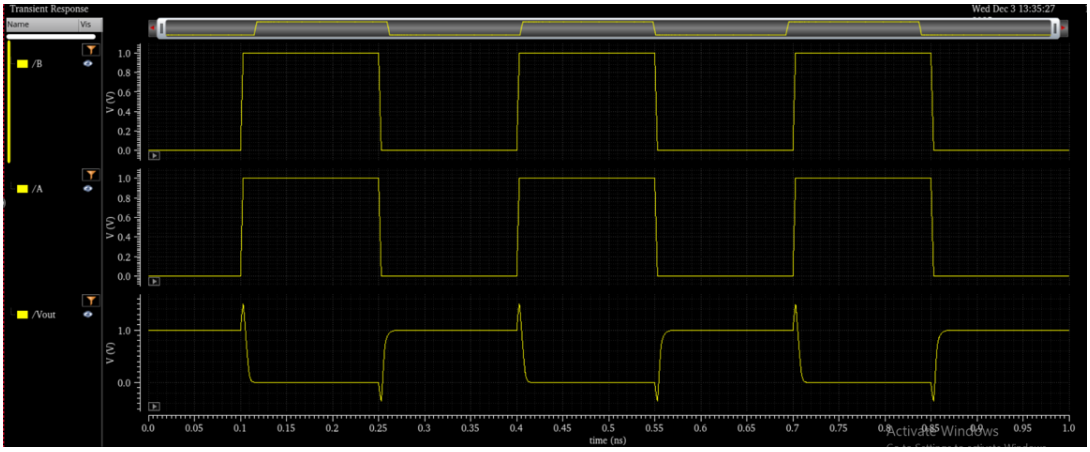


Figure 11: Simulation Waveforms of 2-Input NOR Gate

- **Latency**

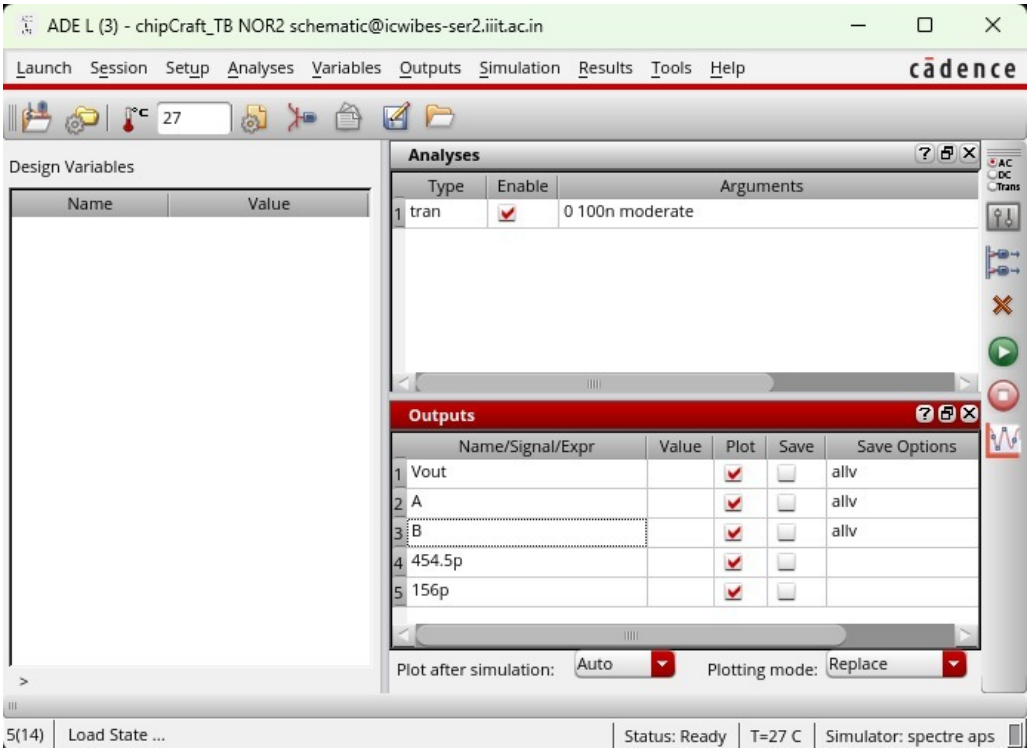
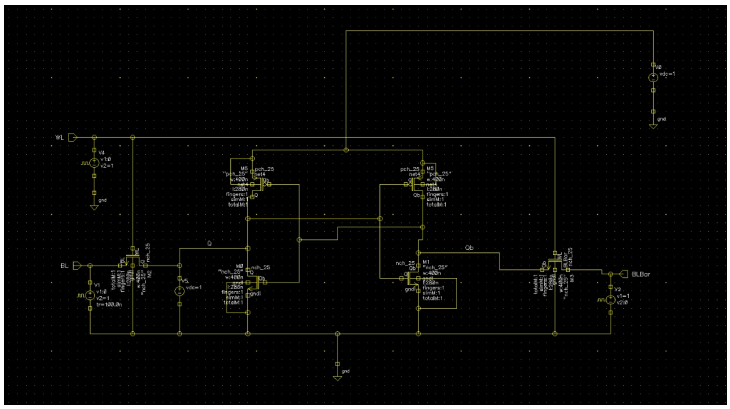
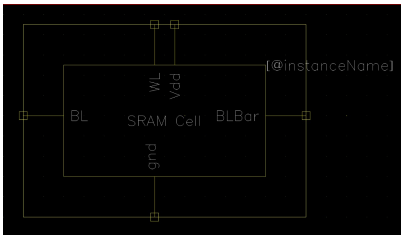


Figure 12: Propagation Delay Characteristics of 2-Input NOR Gate

2.5 6T SRAM Cell



(a) 6T SRAM Cell Schematic



(b) 6T SRAM Cell Symbol

Figure 13: 6T SRAM Bitcell: Schematic and Symbol

3 32x32 SRAM Architecture

4 Adder Design

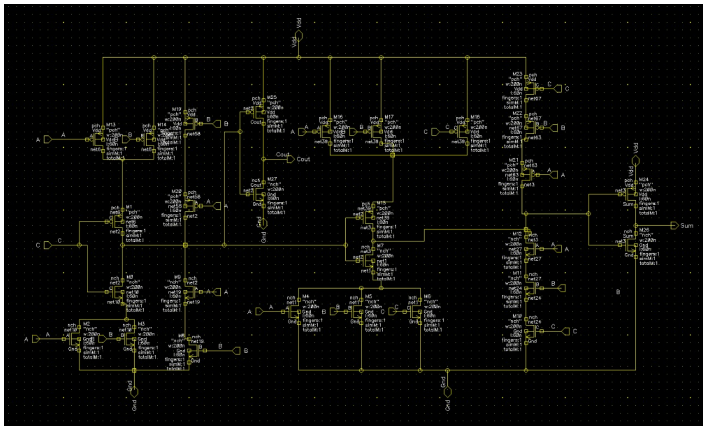
The 5-bit adder used in this design is implemented using a chain of five 1-bit Full Adders, each realized using a mirror-logic CMOS topology.The key advantage of this approach is that both the Sum and Carry outputs are realized using highly symmetrical transistor

networks, which helps reduce skew, offers good noise margins, and ensures predictable delay across process corners.

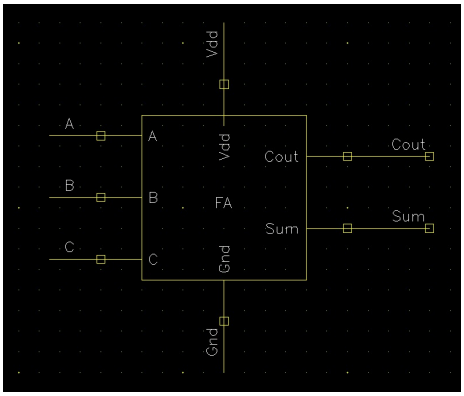
It is required to add two 5-bit addresses to obtain the final address of the SRAM Cell to be accessed. To perform this addition we require 5 full-adders.

4.1 Optimised Design

- **Schematic:**

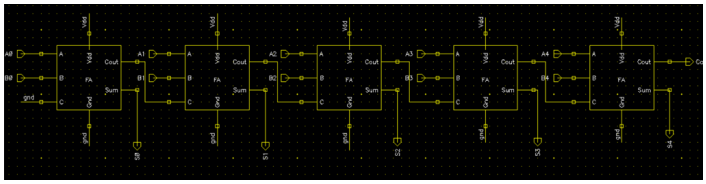


(a) 1-Bit Adder Schematic

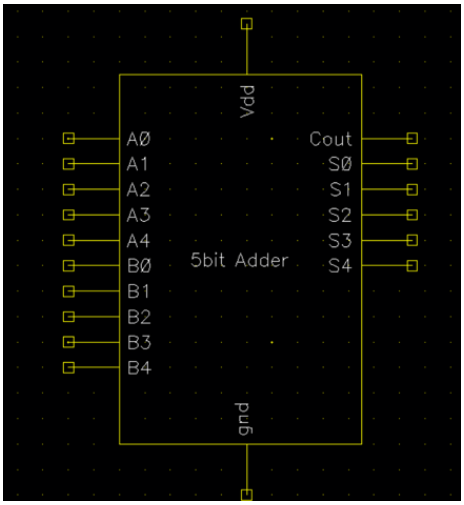


(b) 1-Bit Adder Symbol

Figure 14: 1-Bit Adder: Schematic and Symbol



(a) 5-Bit Adder Schematic



(b) 5-Bit Adder Symbol

Figure 15: 5-Bit Adder: Schematic and Symbol

- **Simluation Results:**

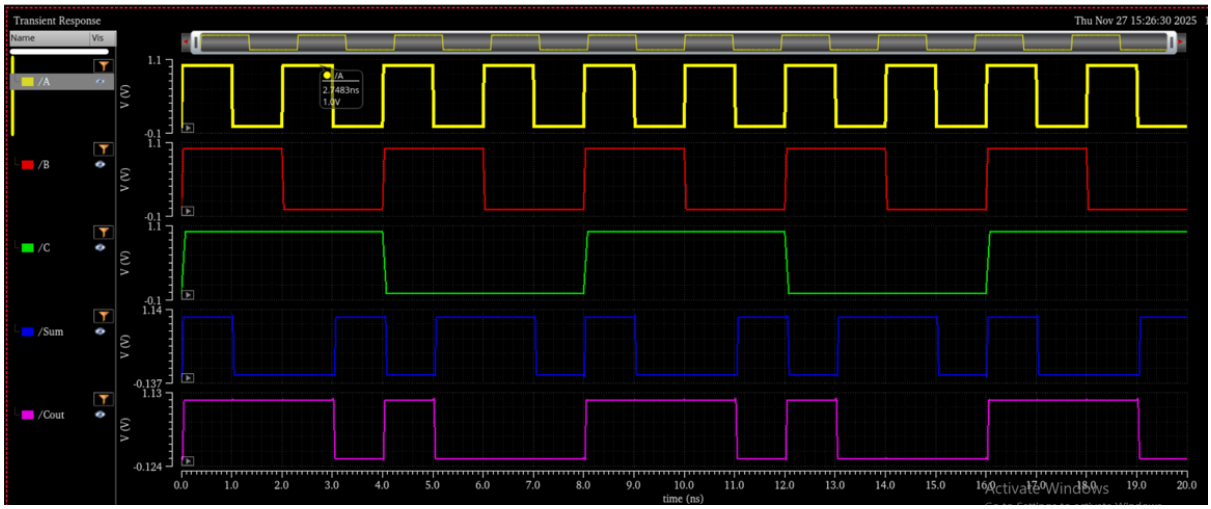


Figure 16: Simulation Waveforms of 1-Bit Adder

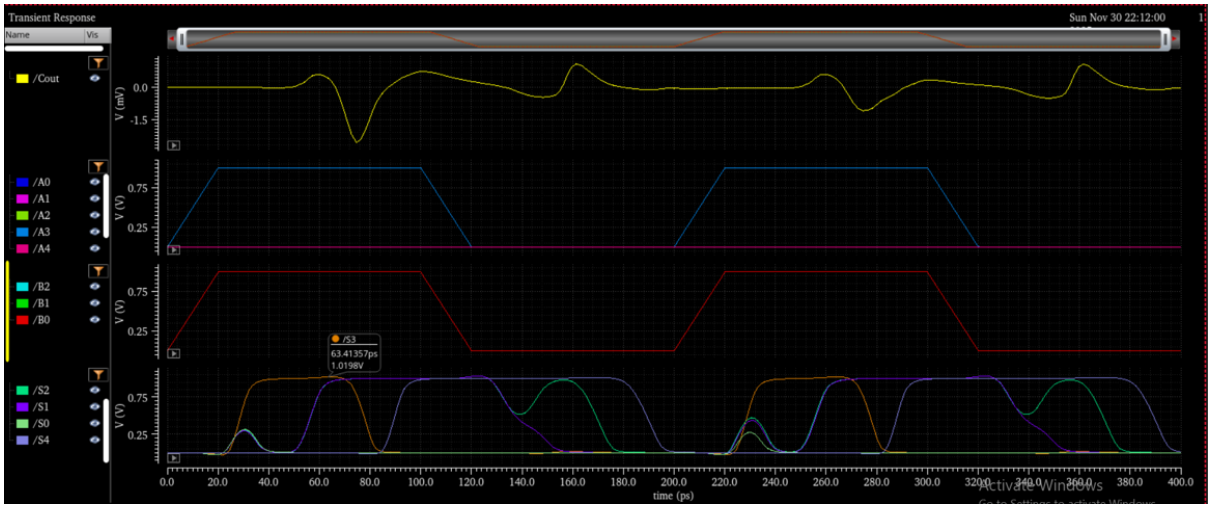


Figure 17: Simulation Waveforms of 5-Bit Adder

- **Critical Path Analysis:**

  Worst case simulation delay to the outputs are noted when all the carries are into toggle. Therefore, 1111(0 - 1) and 00001 values gives the max value.

4.2 Latency and Performance

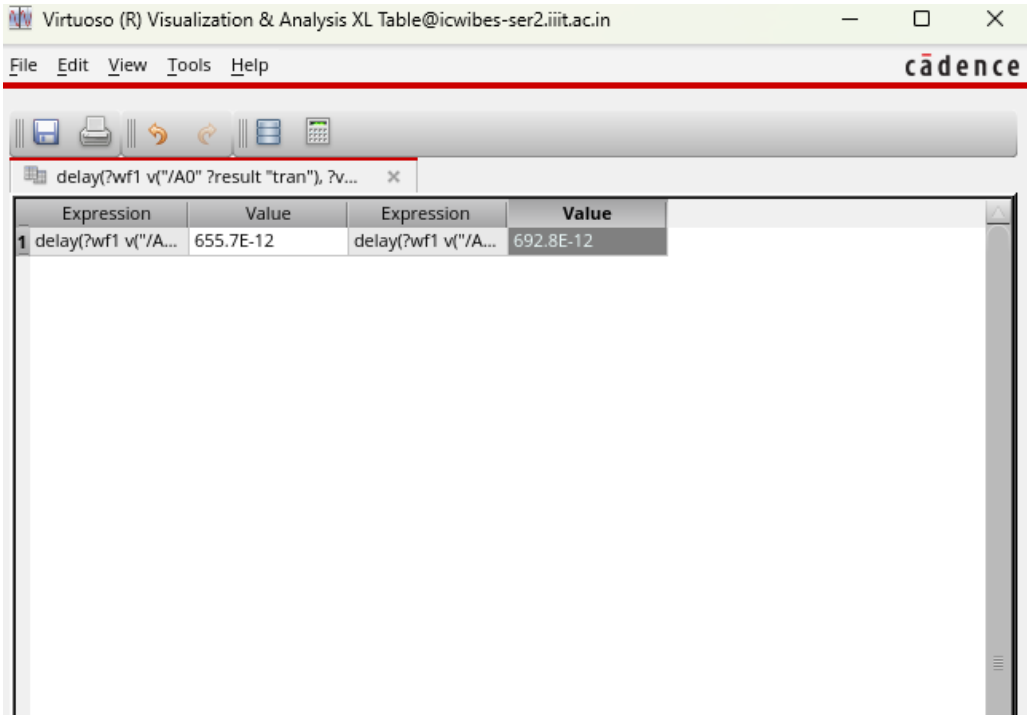


Figure 18: Worst-Case Delay of 5-Bit Full Adder



Figure 19: Fall Delays of 1-Bit Full Adder With Respect to Inputs A and B

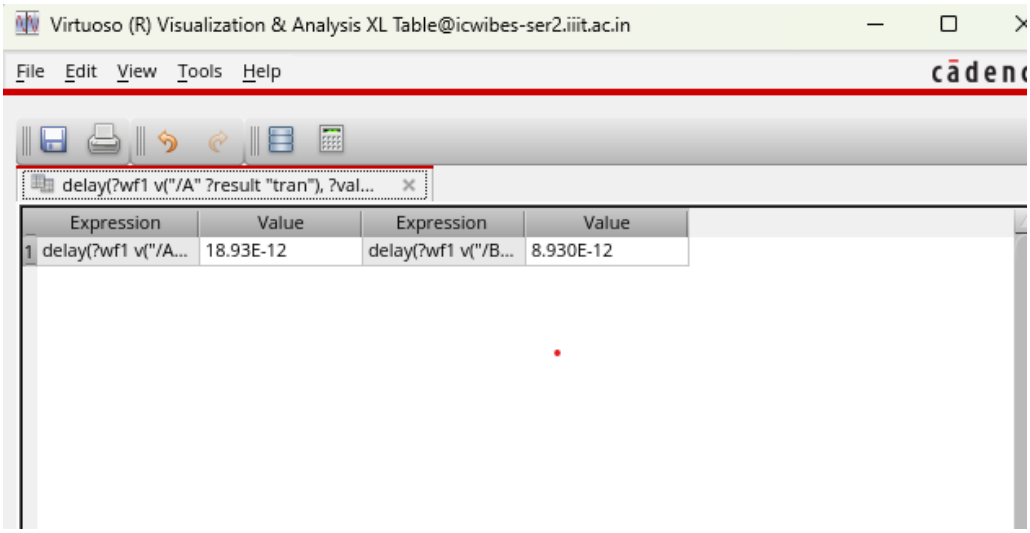
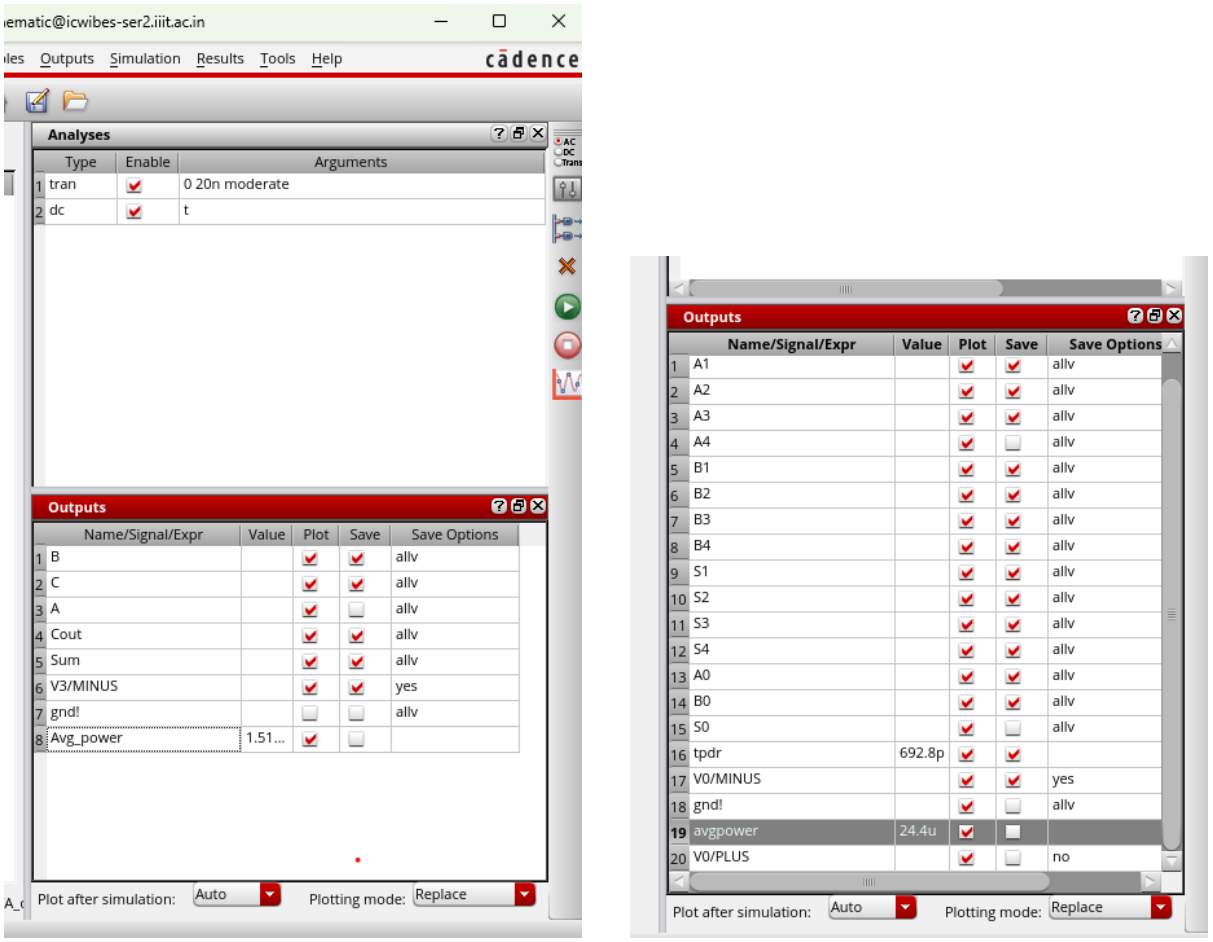


Figure 20: Rise Delays of 1-Bit Full Adder With Respect to Inputs A and B

4.3 Power Consideration



(a) 1-Bit Full Adder

(b) 5-Bit Full Adder

Figure 21: Average Power Consumption Comparison of 1-Bit and 5-Bit Full Adders

4.4 Unoptimized Design

We have done the Adder circuit with PTL logic having only 14 transistors. But the output swing was very low.

Schematic:

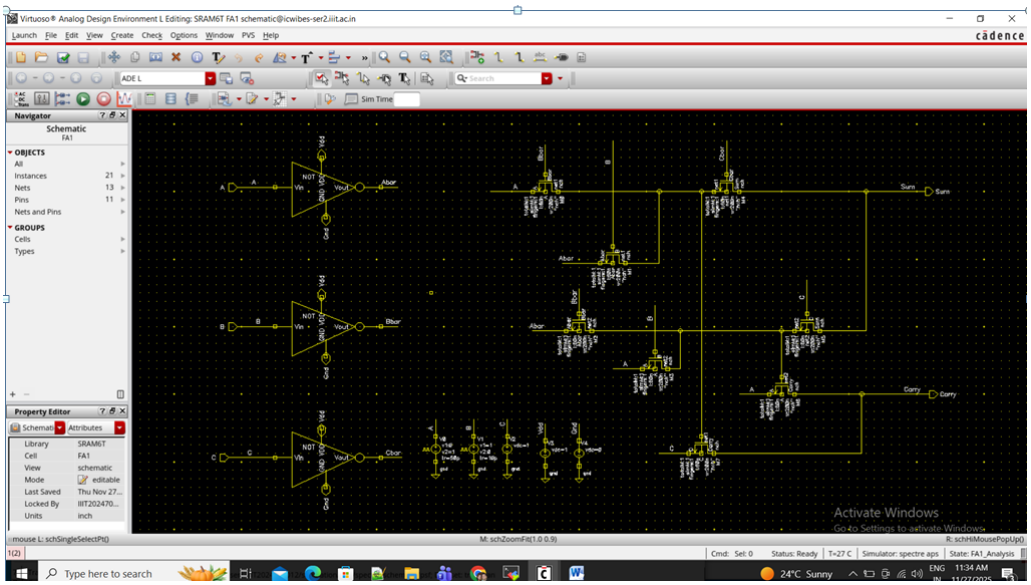


Figure 22: PTL schematic of Full Adder

Waveforms

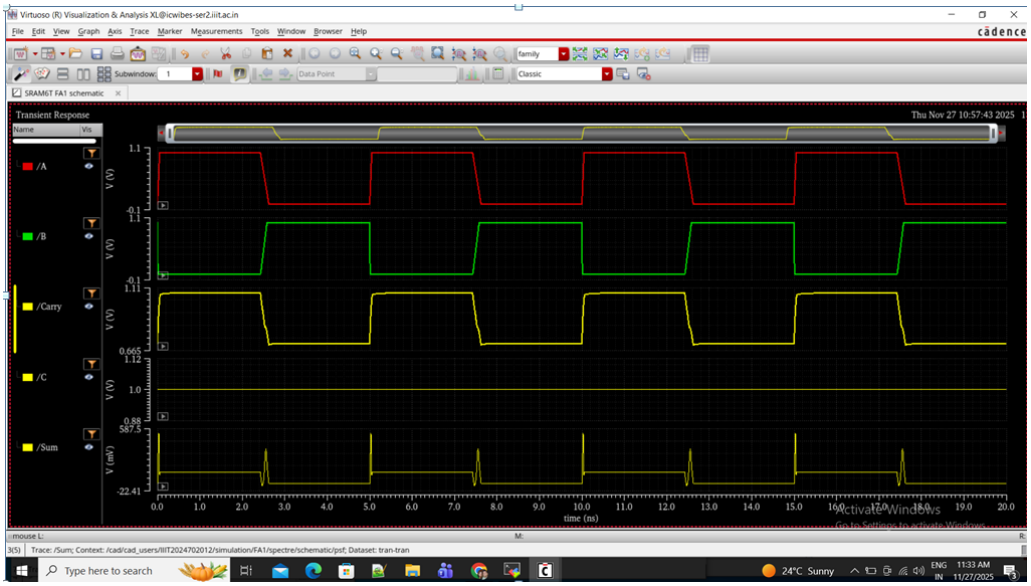


Figure 23: Waveforms of Full Adder – PTL logic

PTL FA is giving degraded logic levels (Carry low 0.7 V, Sum swing 0–0.6 V).That's why using 0.5*VDD as threshold doesn't work. For a proper digital design you'd usually add a restoring inverter at Carry and Sum, so they swing rail-to-rail; then standard 0.5•VDD delay works and noise margins are better.

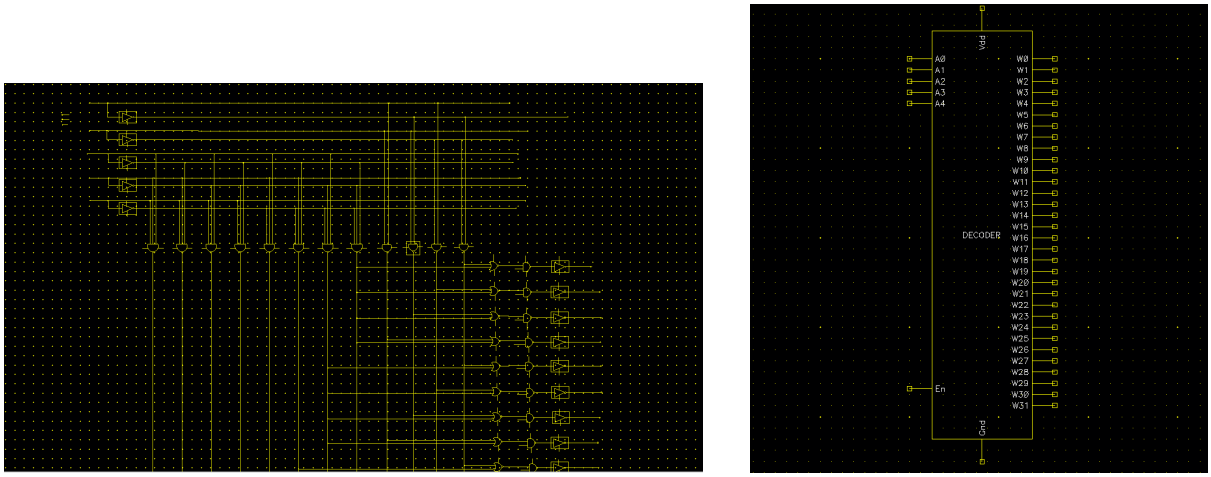
5 Decoder Design

Decoders typically have high electrical and branching effort. Therefore, they need many stages, so the fastest design is the one that minimizes the logical effort. A tree of 2- and 3-input NAND gates and inverters offers the lowest logical effort to build high fan-in gates in static CMOS. In a standard 5-bit decoder, many NAND gates share exactly the same inputs and are thus redundant. The decoder area can be improved by factoring these common NANDs out. This technique is called predecoding. It does not change the path effort of the decoder, but does improve area. In our design we have predecoded 2-bits and 3-bits simultaneously. The outputs are then ANDed to obtain the final decoded results.

5.1 Optimised Design

Description of design and sizing.

- **Schematic:**



(a) 5-to-32 Decoder Schematic (b) Symbol of 5-to-32 Decoder

Figure 24: 5-to-32 Decoder: Schematic and Symbol

- **Simluation Results:**

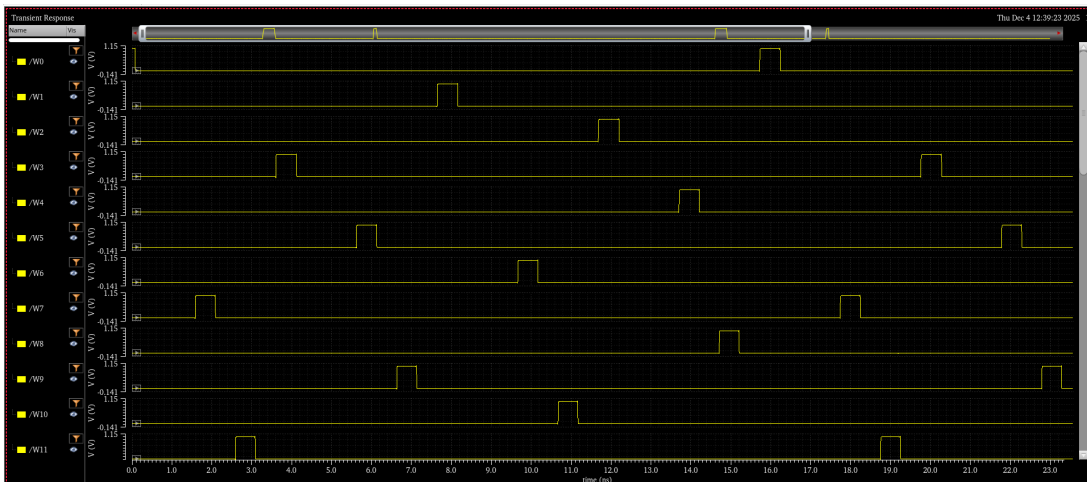


Figure 25: Simulation Waveforms of 5-to-32 Decoder

5.2 Latency and Performance

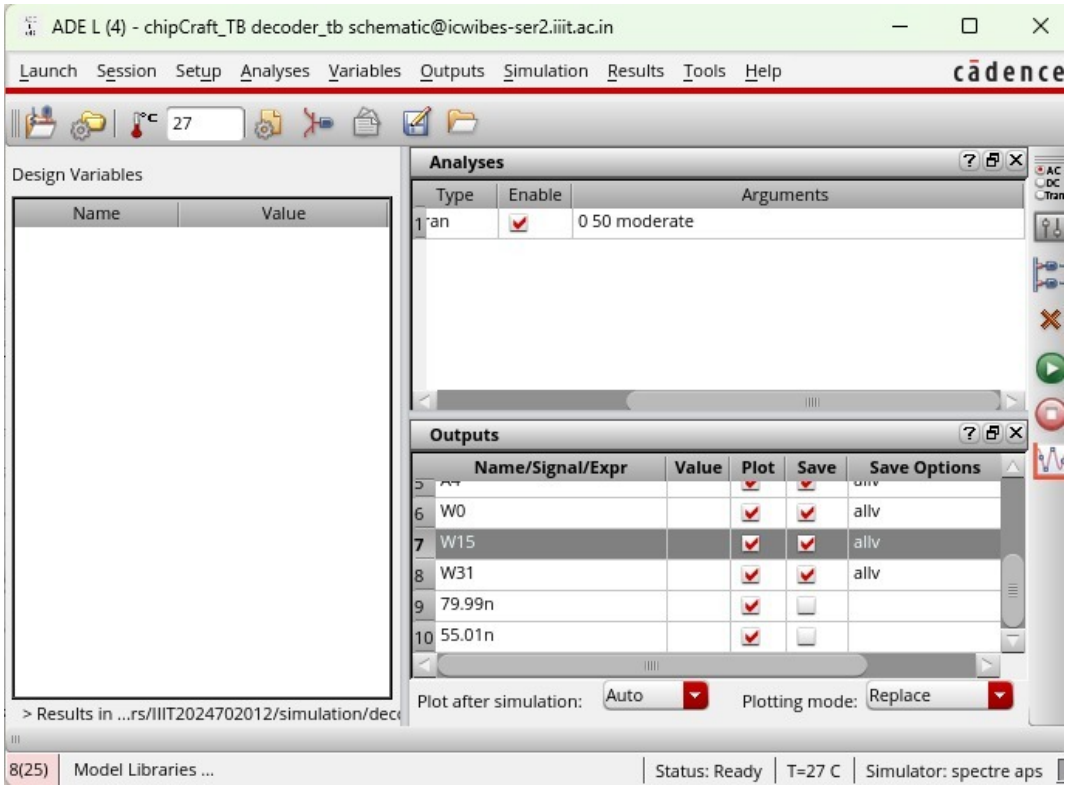


Figure 26: Propagation Delay Characteristics of Decoder Circuit

5.3 Power Consideration

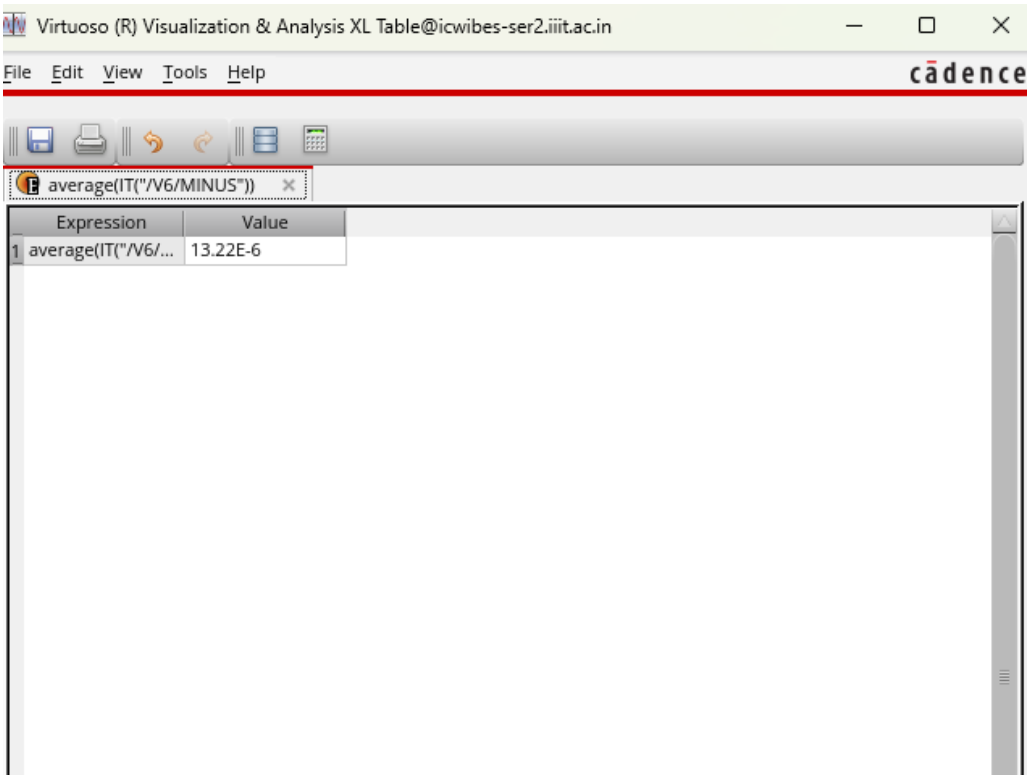


Figure 27: Average Power Consumption of Decoder Circuit

6 32x32 SRAM Array

Row/column organization + bitlines + wordlines. PMOS devices used in the pre-charge is used to bring bitlines to VDD.

The 32x32 SRAM block contains 1024 6T SRAM Cells.

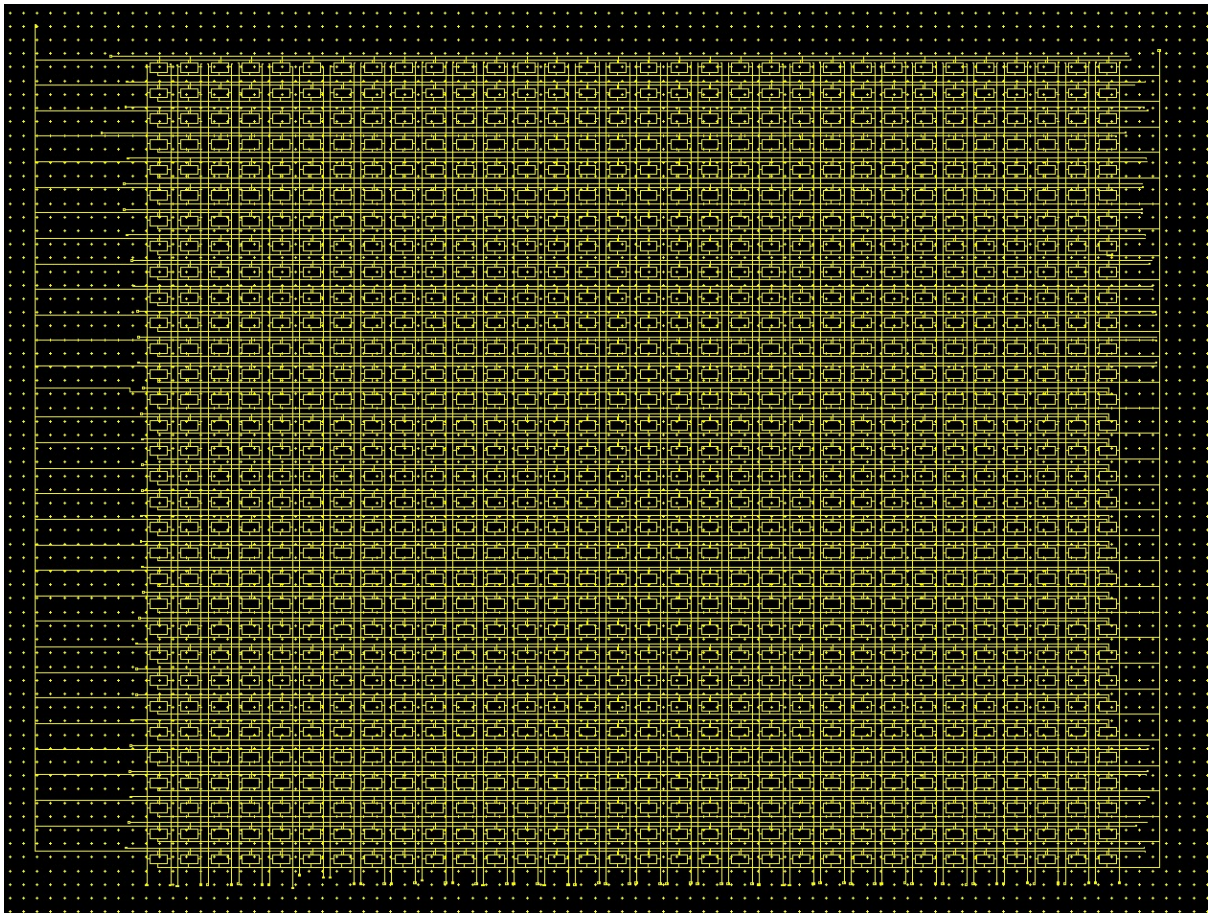
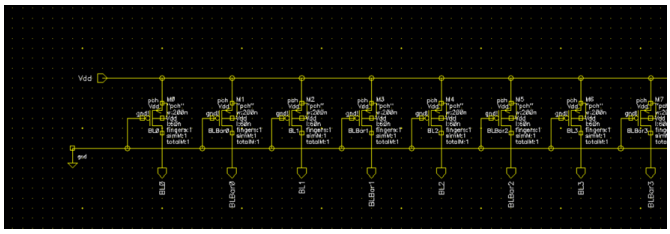


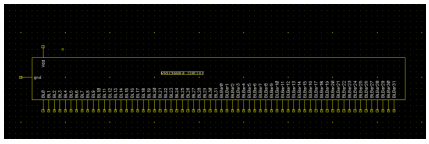
Figure 28: SRAM Architecture Schematic

6.1 Precharge Circuit

The precharge circuit consists of 64 PMOS transistors, 1 connected to each bitline. When the precharge input is low, PMOS switches are closed and all the bitlines charge upto VDD.



(a) Precharge Circuit Schematic



(b) Precharge Circuit Symbol

Figure 29: Precharge Circuit: Schematic and Symbol

6.2 Optimised Design

6.3 6T SRAM Cell Design

The 6T SRAM cell consists of two cross-coupled inverters and two NMOS pass transistors. First, the inverter is characterized in the TSMC 65 nm technology node for equal

noise margins (i.e., switching threshold at $V_{DD}/2$). The value of β obtained from this characterization is then used to size the inverter appropriately for the SRAM cell.

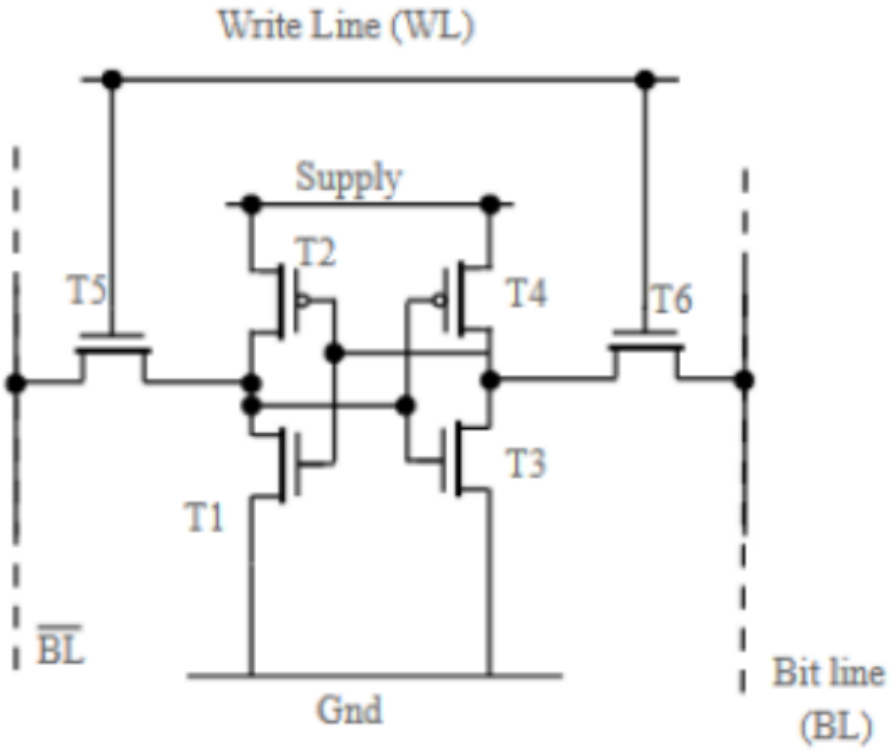


Figure 30: SRAM Architecture Circuit Diagram

6.4 MOSFET Sizing for SRAM Cell

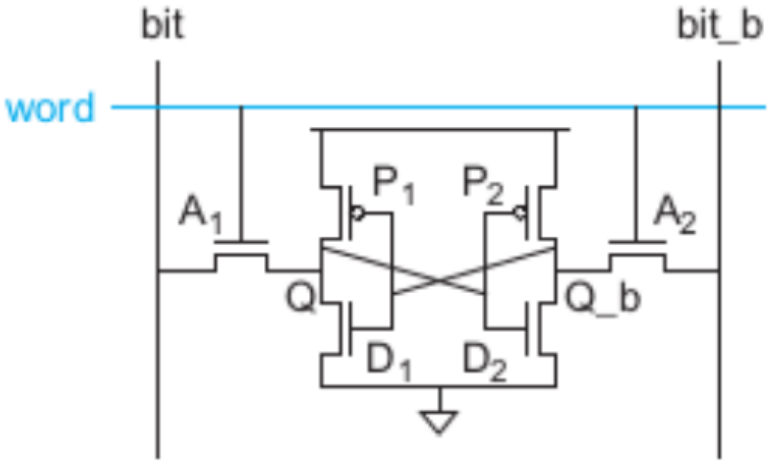


Figure 31: 6T SRAM Cell Schematic Showing Device Labels

During the read operation, the bitlines are precharged to V_{DD}. When the wordline is asserted, the bitline bit should be pulled down through the driver and access transistors (D1 and A1). However, as bit is pulled low, node Q tends to rise. Node Q is held low by the driver transistor D1, but current flowing through A1 attempts to raise it. Therefore, the driver NMOS (D1) must be stronger than the access NMOS (A1). Specifically, the device strengths must be ratioed such that node Q remains below the switching threshold of the $(P2, D2)$ inverter. This constraint is known as *read stability*.

Because of this constraint, the bitline cannot force node Q high through A1. Therefore, writing must be performed by forcing Q_b low through the access transistor A2. The pull-up PMOS (P2) opposes this operation; hence, P2 must be weaker than A2 so that Q_b can be pulled low enough. This constraint is called *writability*. Once Q_b falls sufficiently low, D1 turns off and P1 turns on, pulling Q high and completing the write operation.

The inverter was previously characterized for equal strength of PMOS and NMOS devices. Since the driver NMOS must be stronger than the access NMOS for read stability, the (W/L) of the driver is scaled by a factor of 2. Additionally, because the access NMOS must overpower the pull-up PMOS during a write, the PMOS is kept at minimum width for weakest pull-up strength. The final device dimensions are listed below.

Table 1: MOSFET Dimensions Used in the 6T SRAM Cell

Device	**(W/L)**
Driver MOSFET (NMOS)	(400 n / 60 n)
Access MOSFET (NMOS)	(200 n / 60 n)
Pull-up MOSFET (PMOS)	(200 n / 60 n)

6.5 Read Static Noise Margin

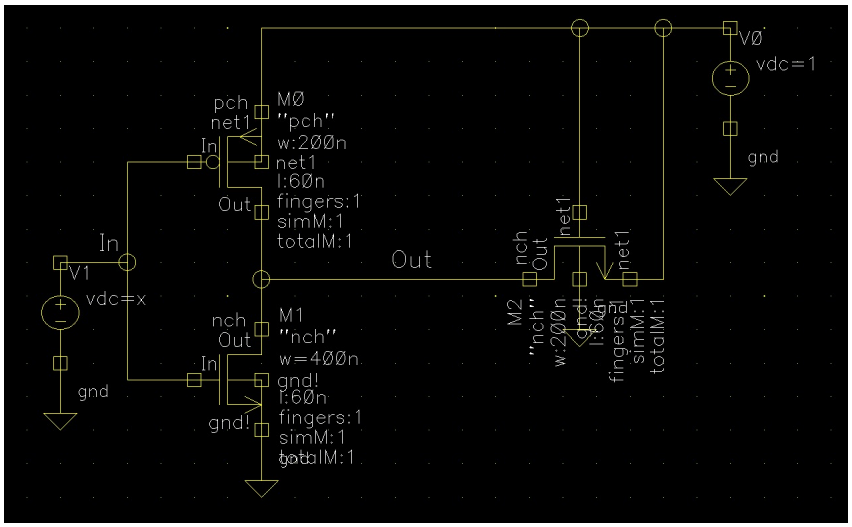


Figure 32: Read Noise Margin Testbench Schematic

The butterfly plots are obtained using the `waveVsWave` function in the Cadence calculator. The exact expressions used are:

1. **V2 vs. V1:**
   ```
   waveVsWave(?x v("/V1" ?result "dc") ?y v("/V2" ?result "dc"))
   ```

2. **V1 vs. V2:**
   ```
   waveVsWave(?x v("/V2" ?result "dc") ?y v("/V1" ?result "dc"))
   ```

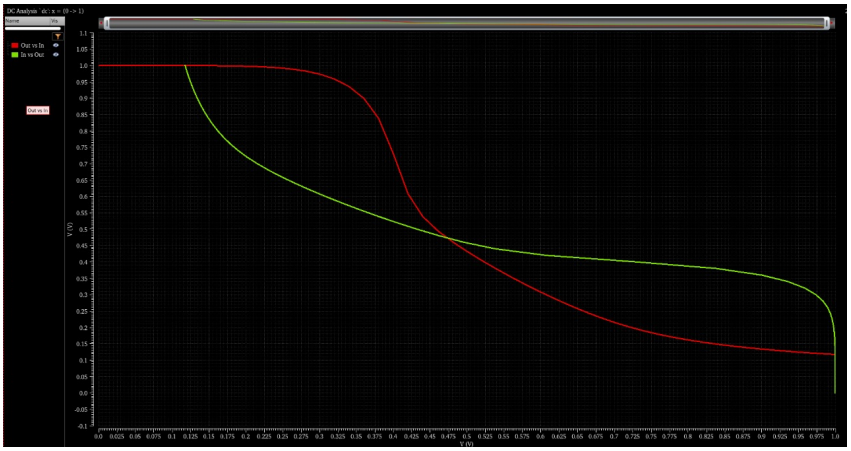


Figure 33: Butterfly Plot for Read Static Noise Margin Extraction

A common method for obtaining the largest square that can fit inside the butterfly curve is to transform the axes by 45°. The maximum vertical separation between the two transformed curves corresponds to the diagonal of the largest square that can fit inside the curve. This is accomplished by plotting the difference between the transformed curves and identifying the maximum value.

Let us consider a general transformation from (x, y) coordinates to (u, v) coordinates rotated by an angle ϕ. The equations governing the transformation are:

$$A_u = A_x \cos\phi + A_y \sin\phi \tag{1}$$

$$A_v = -A_x \sin\phi + A_y \cos\phi \tag{2}$$

Since the butterfly curve is transformed by $-45°$, substituting $\phi = -45°$ gives:

$$A_u = \frac{1}{\sqrt{2}}(A_x + A_y) \tag{3}$$

$$A_v = \frac{1}{\sqrt{2}}(A_x - A_y) \tag{4}$$

In Cadence Virtuoso, this transformation is implemented using the following expressions:

1. **Transformed V2 vs. V1:**

```
waveVsWave(?x ((1 / sqrt(2)) * (v("/V1" ?result "dc") -
                                v("/V2" ?result "dc")))
          ?y ((1 / sqrt(2)) * (v("/V1" ?result "dc") +
                                v("/V2" ?result "dc"))))
```

2. **Transformed V1 vs. V2:**

```
waveVsWave(?x ((1 / sqrt(2)) * (v("/V2" ?result "dc") -
                                v("/V1" ?result "dc")))
           ?y ((1 / sqrt(2)) * (v("/V2" ?result "dc") +
                                v("/V1" ?result "dc"))))
```

Conclusions from the Read Margin Analysis

1. The diagonal of the largest inscribed square obtained from the transformed butterfly plot is found to be:
$$d = 235.76\,\text{mV}$$

2. The side length of the square is:
$$a = \frac{1}{\sqrt{2}} \times 235.76\,\text{mV} = 166.71\,\text{mV}$$

3. The Read Static Noise Margin (RSNM) of the SRAM cell is therefore:
$$\text{RSNM} = 166.63\,\text{mV}$$

4. The worst-case voltage rise in the SRAM cell during a read operation (i.e., value of V_2 when $V_1 = V_{DD}$), marked by `M19`, is:
$$V_{2,\text{rise}} = 161.11\,\text{mV}$$

6.6 Write Static Noise Margin

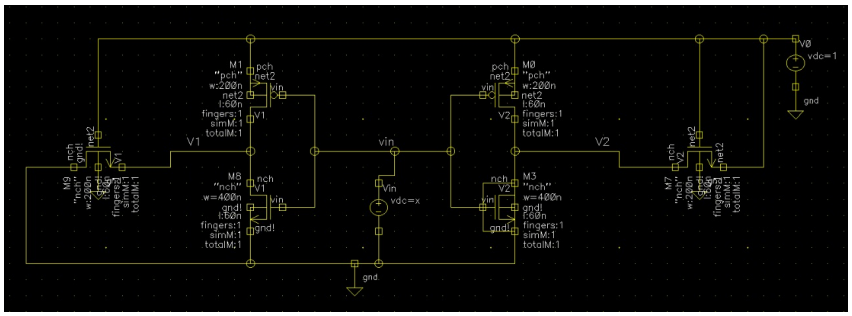


Figure 34: Write Noise Margin Testbench Schematic

The DC analysis setup is identical to that used for the Read Static Noise Margin (RSNM) extraction. The butterfly plots for Write SNM are generated using the `waveVsWave` function in the Cadence Virtuoso calculator. The exact expressions used are:

1. **Vin vs. V1:**
   `waveVsWave(?x v("/V1" ?result "dc") ?y v("/Vin" ?result "dc"))`

2. **V2 vs. Vin:**
   `waveVsWave(?x v("/Vin" ?result "dc") ?y v("/V2" ?result "dc"))`

To obtain the largest square inside the butterfly curve, the same coordinate transformation used for RSNM is applied here. In Cadence Virtuoso, this is implemented using the following expressions:

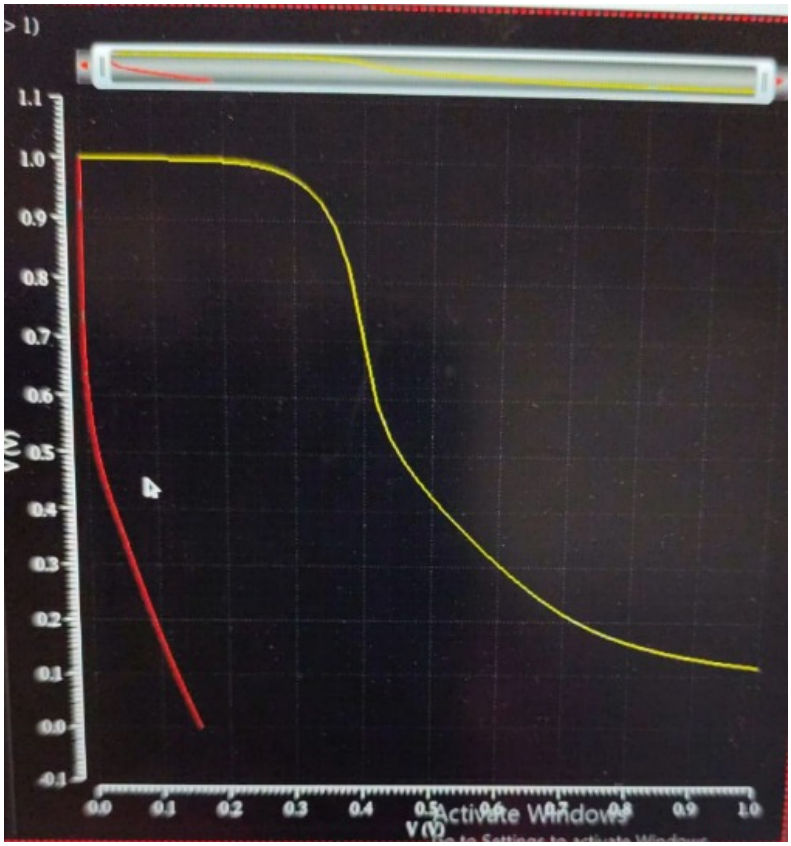


Figure 35: Write Noise Margin (WNM) Waveform of 6T SRAM Cell

1. **Transformed Vin vs. V1:**

```
waveVsWave(?x ((1 / sqrt(2)) * (v("/V1" ?result "dc") -
                                v("/Vin" ?result "dc")))
           ?y ((1 / sqrt(2)) * (v("/V1" ?result "dc") +
                                v("/Vin" ?result "dc"))))
```

2. **Transformed V2 vs. Vin:**

```
waveVsWave(?x ((1 / sqrt(2)) * (v("/Vin" ?result "dc") -
                                v("/V2" ?result "dc")))
           ?y ((1 / sqrt(2)) * (v("/Vin" ?result "dc") +
                                v("/V2" ?result "dc"))))
```

Conclusions from the Write Margin Analysis

1. The diagonal of the largest inscribed square obtained from the transformed butterfly plot is found to be:

$$d = 506.635\,\text{mV}$$

2. The corresponding side length of the square is:

$$a = \frac{1}{\sqrt{2}} \times 506.635\,\text{mV} = 358.25\,\text{mV}$$

3. The Write Static Noise Margin (WSNM) of the SRAM cell is therefore:

$$\text{WSNM} = 357.67\,\text{mV}$$

4. The worst-case cell voltage during a write operation (found by measuring V_1 when $V_2 = 0$), indicated by marker `M5`, is:

$$V_{1,\text{worst}} = 176.29\,\text{mV}$$

7 Sense Amplifier Design

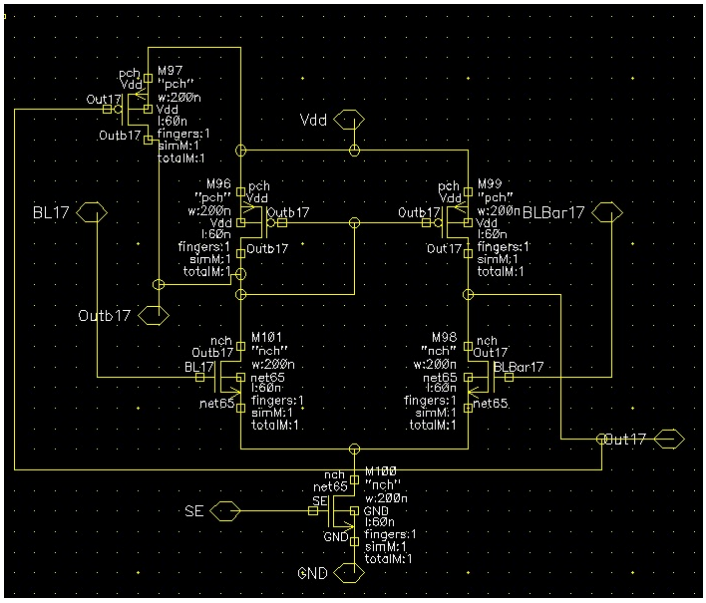
The sense amplifier receives the differential bitline signals and an enable input (SE). During a read, when SE is asserted high, the bitline voltages are sensed and amplified to generate rail-to-rail digital outputs.

Mirror-current differential sense amplifier: An NMOS differential input pair is connected to BL and BLB, with a tail current source controlled by SE. The differential current produced by the input pair, $\Delta I = g_m \cdot \Delta V_{BL}$, is converted into output voltage using PMOS current-mirror loads. The output voltage is approximately proportional to the product of differential current and output resistance, $V_{out} \approx \Delta I \cdot R_{out}$.

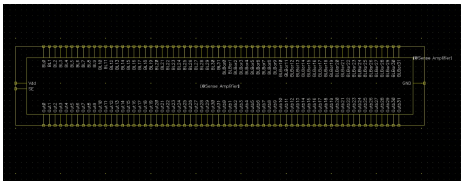
PMOS mirror devices are sized using the inverter sizing ratio obtained from SRAM cell characterization ($\beta = 2.35$), ensuring balanced rise/fall characteristics and equal noise margins. Input NMOS widths are selected to balance bitline loading and transconductance, while the tail and enable devices are upsized to reduce turn-on delay. The design is verified through transient simulations, Monte Carlo analysis (offset), and PVT corner checks.

7.1 Design

- **Schematic and Symbol:**



(a) Sense Amplifier Schematic



(b) Sense Amplifier Symbol

Figure 36: Sense Amplifier: Schematic and Symbol

- **Operation:** When BLB = 1, BL = 0, and SE = 1, the output resolves to Out = 0 and OutB = 1. Conversely, for BLB = 0, BL = 1, the output resolves to Out = 1 and OutB = 0.
- **Critical Path:** The main delay path is from the bitline input transition (BL/BLB) to the output nodes. Delay is evaluated for both BL and BLB switching conditions.

7.2 Simulation Results and Delay Calculations

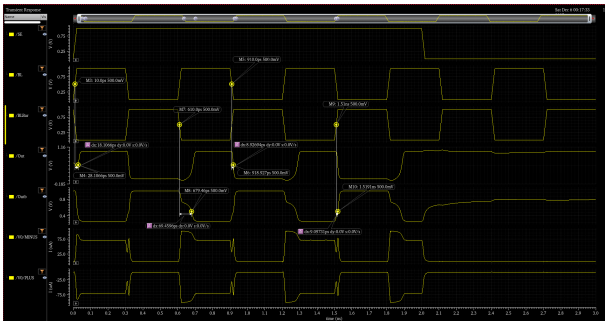


Figure 37: Delay of the simple 1-bit circuit

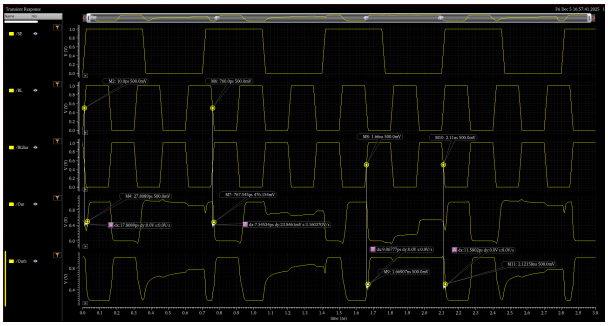


Figure 38: Delay of the optimised 1-bit circuit

tpd	Simple Circuit (ps)	Optimised Circuit (ps)
tpd(BL)	13.51	12.677
tpd(BLB)	39.278	10.324

Table 2: Delay values of the Sense Amplifier

A significant reduction in delay is observed, with the worst-case delay improving from 39.278 ps to 10.324 ps. The final critical delay of the sense amplifier including enable and regeneration is measured as **189.59 ps**.

7.3 Power Consideration

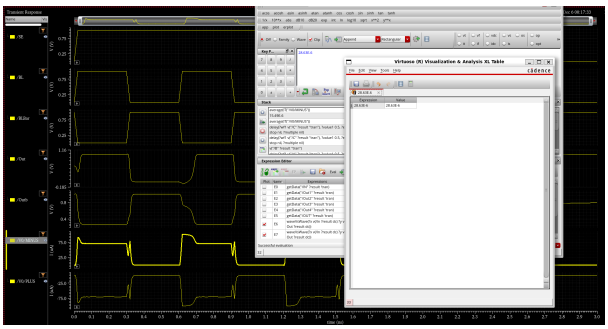


Figure 39: Power of simple 1-bit Sense Amplifier

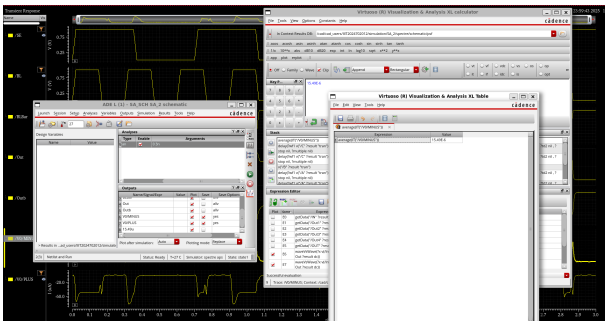


Figure 40: Power of optimised 1-bit Sense Amplifier

Power	Simple Circuit (μW)	Optimised Circuit (μW)
Supply Power	25.63	15.49

Table 3: Power consumption comparison

7.4 Trade-offs

Reducing delay requires stronger devices, which increases dynamic power. Likewise, lowering power by reducing device sizes increases delay. The optimised design achieves an improved balance, with both lower delay and reduced power compared to the initial circuit.

8 Complete 32x32 bit SRAM Design

8.1 Optimised Complete Design

- **Schematic:**

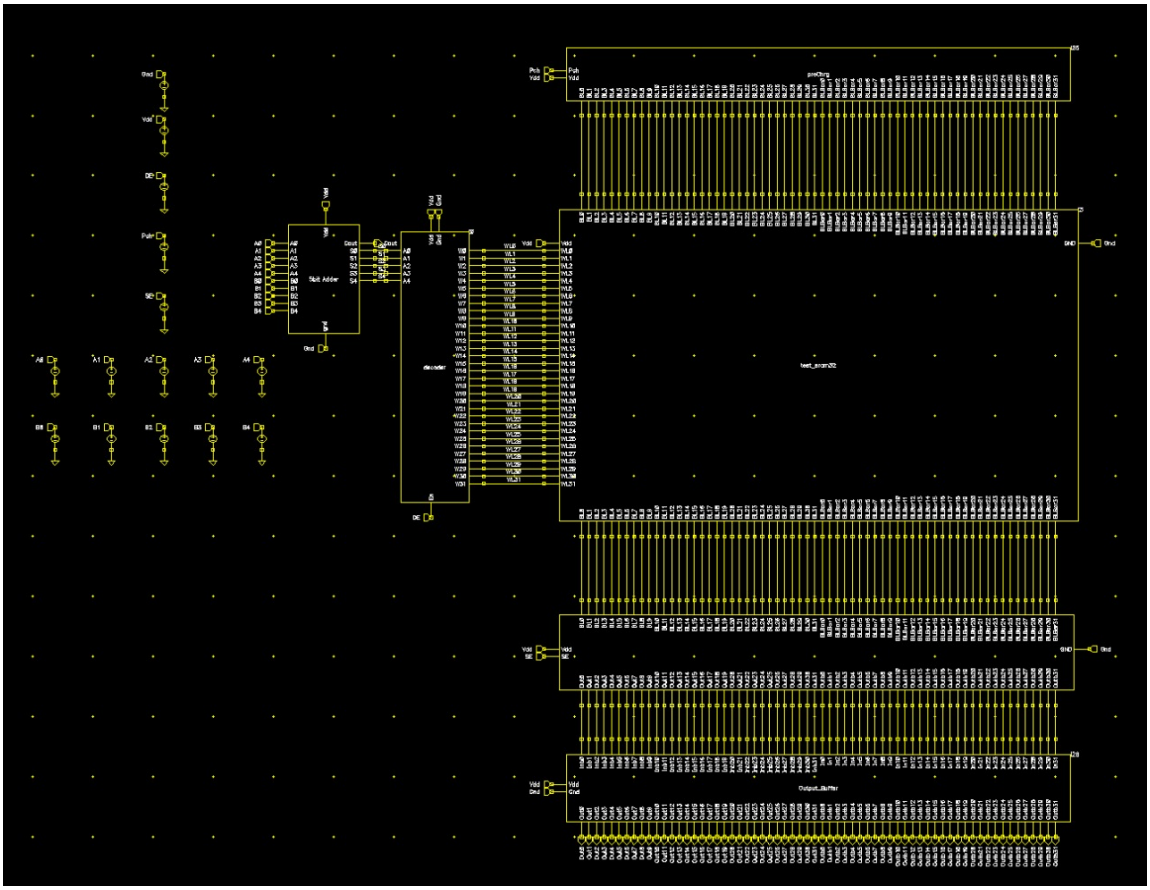


Figure 41: 32×32 SRAM Schematic

- **Simluation Results:**

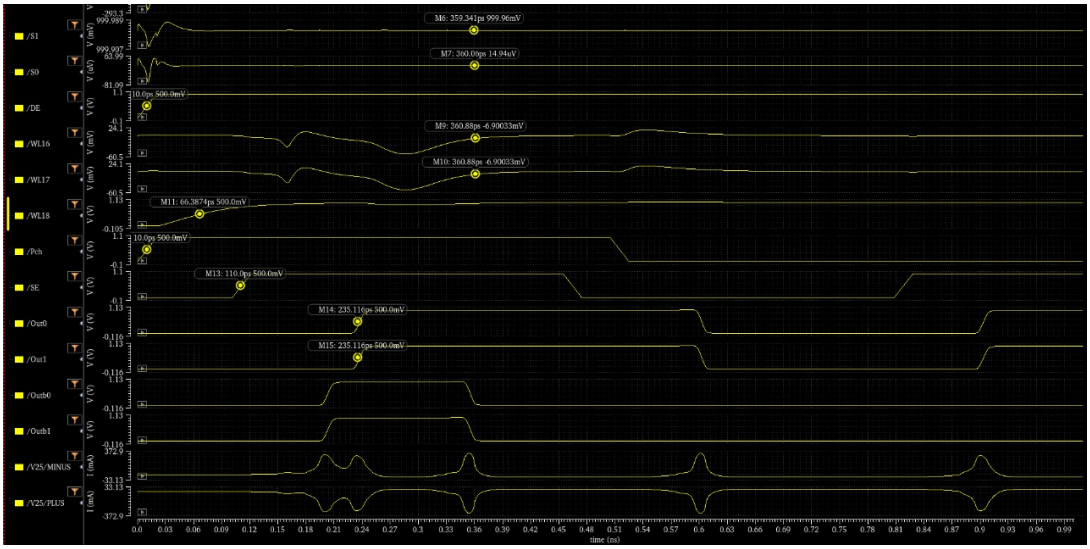


Figure 42: Simulation Waveforms of 32×32 SRAM Architecture

- **Critical Path Analysis:**

8.2 Latency and Performance

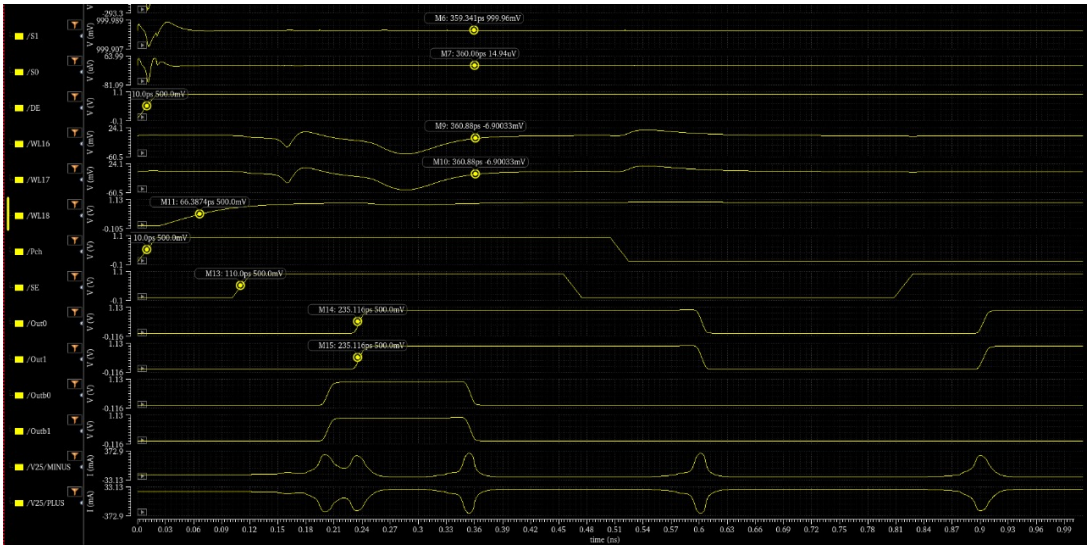


Figure 43: Delays of 32×32 SRAM Architecture

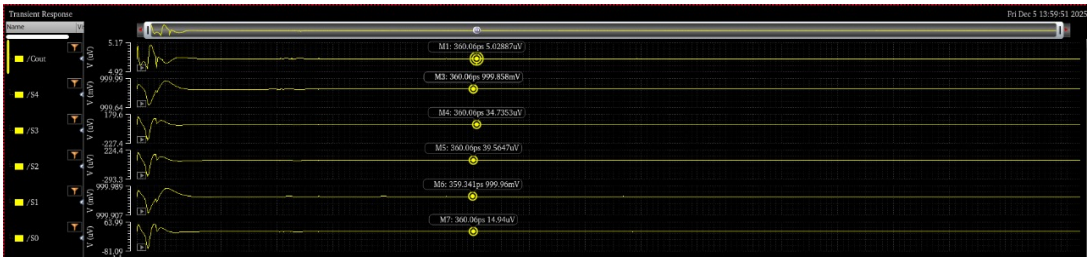


Figure 44: Delays of 32×32 SRAM Architecture

The read operation takes approx 360ps.

8.3 Power Consideration

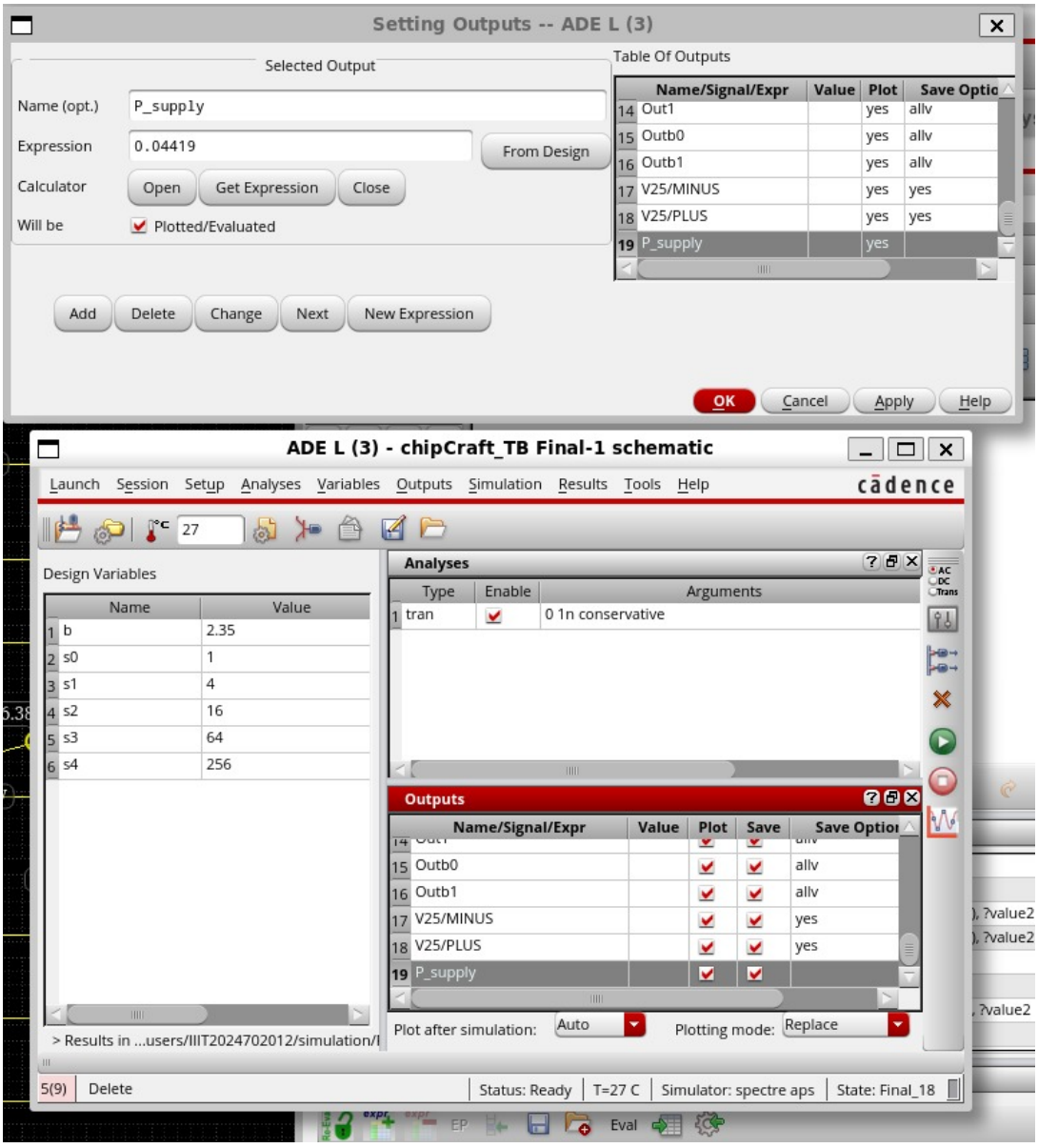


Figure 45: Average Power Consumption of 32×32 SRAM Architecture

9 Conclusion

In this project, a complete 32×32-bit SRAM architecture was designed, optimized, and validated using the TSMC 65 nm technology node. All fundamental building blocks—including the 5-bit adder, predecoded 5-to-32 decoder, 6T SRAM cell, precharge circuitry, and sense amplifier were carefully sized and analyzed to achieve reliable memory operation.

Simulation results confirm that the design meets required speed, stability, and power targets. The SRAM achieves a read delay of approximately 360 ps, while maintaining robust Read SNM (166 mV) and Write SNM (358 mV), ensuring stable functionality un-

der typical conditions. Power characterization shows that the optimized circuits provide improved efficiency without compromising performance.

Overall, the project successfully demonstrates a fully functional and optimized SRAM subsystem,